Group Assignment

IB9JH: Programming for Quantitative Finance

Hema Srikar Ankem	5716485
Alexandre Banh	2202631
Bobby Wong	5709762
Aradhana Krishna Rao	2212328
Joanna Okenla	2200445

Module Instructor: Aurash Karimi

Warwick Business School
University of Warwick

April 8, 2026

WARWICK
THE UNIVERSITY OF WARWICK

Contents

Introduction **1**

1 Project Overview **1**
1.1 Project Selection . . . 1
1.2 Project Objectives . . . 1
1.3 Project Documentation . . . 1
1.4 Task Allocation and Team Strengths . . . 2

2 Methodology **3**
2.1 Theoretical Modelling Approach . . . 3
2.2 Modular Structure . . . 4
2.3 Object-Oriented Programming Structure . . . 4
2.4 Memory Management and Optimization . . . 7

3 Testing **7**
3.1 Test Coverage . . . 7

4 LSM Pricer Visual Analysis for Put Option **8**
4.1 Simulated Paths: ITM & OTM and Decision Map . . . 8
4.2 LSM Regression: Continuation Value . . . 9

5 Benchmark and Convergence Results **10**
5.1 Validation . . . 10
5.2 Convergence Analysis . . . 11

6 Project Limitations **14**
6.1 Further Scopes . . . 14

Conclusion **15**

A Additional Task Allocation Details **17**

B Libraries **20**

C Object Oriented Programming Structure **21**
C.1 Option Payoff . . . 21
C.2 Basis Functions . . . 22
C.3 Underlying SDE . . . 25
C.4 OLS Regression . . . 26
C.5 LSM Pricer . . . 28
C.6 Convergence Analyser . . . 31

D Implementation Issues **33**
D.1 Eigen Package . . . 33
D.2 Compiling Times . . . 33
D.3 GitHub Upstream link . . . 34

E Variables Types, C++ Language Features and Structures **34**

E.1 constexpr . . . 34
E.2 auto . . . 35
E.3 static_cast . . . 35
E.4 Explicit Constructors . . . 36
E.5 Override Specifier . . . 36
E.6 Deleted Copy Operations . . . 36
E.7 Reserve for Performance . . . 36

List of Tables

1.1 Project Documentation Overview . . . 1
1.2 Work Distribution . . . 2
3.1 Coverage report generated by `run_coverage.sh` . . . 7
5.1 American put prices from BS (European), FD, and LSM pricers across 12 parameter combinations ($K = 40$, $r = 0.06$). Early exercise premium = LSM $-$ BS. . . . 10
A.1 Task Details . . . 19

List of Figures

2.1 OOP architecture of the LSM pricing framework. Dashed nodes denote abstract interfaces, solid nodes denote derived classes or supporting modules, blue arrows denote inheritance relationships, and orange arrows denote usage or ownership within the pricing engine. . 5
4.1 In the Money GBM paths . . . 8
4.2 Decision Map of Simulated Paths . . . 8
4.3 Fitted Continuation Curve . . . 9
4.4 Exercise Continuation Curve . . . 10
5.1 Absolute error |LSM $-$ FD| vs number of simulated paths N on a log-log scale. . . . 11
5.2 Absolute error |LSM $-$ FD| vs number of exercise dates M on a log-log scale. . . . 12
5.3 Absolute error |LSM $-$ FD| as a function of polynomial basis order p. . . . 12
5.4 LSM prices across 10 random seeds ($N = 10{,}000$, $M = 50$, $p = 3$). Dashed line shows the mean; shaded band shows $\pm 2\sigma$. . . . 13
5.5 Per-seed standard error: plain vs antithetic. Variance reduction from antithetic variates across 10 seeds. . . . 13
5.6 Average standard error summary. Variance reduction from antithetic variates across 10 seeds. . . . 14

WARWICK
THE UNIVERSITY OF WARWICK

Abstract

This project implements a Monte Carlo-based framework for pricing American-style options using the Longstaff-Schwartz Method (LSM)[1], a widely adopted least-squares regression approach that approximates the optimal early exercise boundary through backward induction. The core challenge in American option pricing lies in determining when early exercise is optimal, a problem that defies closed-form solution in most settings. LSM addresses this by regressing discounted future cash flows against a set of basis functions evaluated at the current asset price, yielding an estimate of the continuation value at each exercise date.

The implementation is written in C++ and structured around several modular components, a stochastic process layer supporting Geometric Brownian Motion, and a jump-diffusion model with Poisson killing jumps. The flexible payoff interface accommodates both put and call option. A basis function library offering both Laguerre polynomial and monomial expansions. An OLS regression engine built on Eigen's QR decomposition. The Antithetic variates are incorporated as an optional variance reduction technique. A convergence analyzer to evaluates the sensitivity of the estimated option price to the number of simulation paths, the number of exercise dates, and the order of the regression basis, with results benchmarked against Black-Scholes European prices and finite difference American prices.

The framework is designed with extensibility in mind, allowing alternative stochastic processes, payoff structures, and basis sets to be introduced with minimal modification to the core pricing engine.

Keywords: LSM Pricer, Option Payoff, Ordinary Least Square Regression, Geometric Brownian Motion, Jump Diffusion Process, Black-Scholes Model, Finite-Difference Analysis, Object Oriented Programming, Abstract, Inheritance, Polymorphism, Encapsulation, Dynamic Memory Allocation, RAII principles

***Note**: The report word limit does not include the characters/words from the UML diagrams, mathematical formulas or notations, tikz figures/flowcharts, plots, code snippets, abstract, and the appendix. The main body of our report, from the Introduction to the Conclusion page, is 1,995 words in total.*

Artificial Intelligence Declaration

AI-assisted tools, namely Claude Sonnet 4.6 and ChatGPT 5.3, were used in a limited capacity for peripheral tasks, such as: Python graph formatting for convergence, generating a simple Bash script to run testing coverage, while some members relied on AI for debugging minor syntax/environment issues, namely interpreting runtime/compiler errors and to generate UML diagram in Section 2.3 on Page 6. These tools were only used to support minor tasks, all core modelling, implementation, and analysis were carried out independently and verified via testing, benchmarking and literature.

List of Abbreviations

ATM At-The-Money

GBM Geometric Brownian Motion

ITM In-The-Money

LSM Least Square Monte-Carlo Simulation

OLS Ordinary Least Square

OOP Object Oriented Programming

OTM Out-Of-The-Money

Introduction

The valuation of American options is a fundamental problem in finance, because the possibility of early exercise precludes a closed-form solution and necessitates numerical methods. Longstaff-Schwartz [1] addresses this through the Least Square Monte-Carlo Simulation (LSM) algorithm, which estimates continuation values by least-squares regression on simulated paths and provides a simulation-based framework for pricing. This is particularly attractive when finite-difference techniques become restrictive.

1 Project Overview

1.1 Project Selection

Several project directions were considered before selecting the Longstaff-Schwartz [1] approach. This emerged as the strongest choice as it combines mathematical finance, numerical analysis, and object-oriented software design in line with the assignment aims and the group's strengths. It provided a financially substantive problem with clear benchmark comparisons. Its simulation-intensive structure lends itself to a modular C++ implementation, benefiting from compiled efficiency and reproducible performance.

1.2 Project Objectives

This project aims to implement the LSM algorithm in C++ and evaluate its accuracy and convergence in pricing American put options under Geometric Brownian Motion (GBM). The implementation is validated against finite-difference and Black-Scholes benchmarks, with analysis of convergence with respect to the number of simulated paths, exercise dates, and regression basis functions. A further objective is to develop an extensible framework, supported by modular class design, RAII-based memory management, and unit testing, so alternative processes, payoffs, and basis specifications can be incorporated without altering the core logic.

1.3 Project Documentation

Basis Functions	*Option Payoff*	*OLS Regression*	*Underlying SDE*
View	View	View	View
LSM Pricer	*Convergence & Benchmark*	*Tests*	UML Diagram
View	View	View	View

Table 1.1: *Project Documentation Overview*

Documentation for each component is hosted on GitHub Pages. Regular updates alongside code changes supported efficient workflow and facilitated independent work across the project. For convenience and quick deployment, the open-source Jekyll framework with the "Just the docs" theme was used. [2].

> **GitHub Pages** The project documentation is hosted online and can be accessed at: LSM American Option pricer - GitHub page.
>
> The GitHub pages are an **extension of this report**, which contains detailed explanation of classes, functions, and outputs of the project files. It also has extensive details of the test cases used to validate the functions' output.

1.4 Task Allocation and Team Strengths

For GitHub, version control was managed by Srikar, who reviewed and merged each member's contributions into the main branch throughout development.

Table 1.2: *Work Distribution*

Team Member	Tasks	Strengths and Experience
Srikar	`basis_functions.cpp`, `basis_functions.hpp`, `lsm_pricer.cpp`, `lsm_pricer.hpp`, `main.cpp`, `CMakeLists.txt`, `Analysis_plots.ipynb`	Prior C++ experience supported efficient project structuring and pricing engine integration.
	Wiki page, GitHub Pages, Git and Repository Management, Project Structuring, Report Writing	Version-control expertise enabled repository management, GitHub Pages deployment, and report coordination.
Alex	`ols_regression.cpp`, `ols_regression.hpp`, `finite_difference.cpp`, `finite_difference.hpp`	Mathematical and algorithms experience supported the QR-based regression and implicit finite-difference scheme, including the Brennan–Schwartz solver and boundary conditions.
	Wiki page, Report Writing	Responsible for "OLS Regression" and "MC Paths" wiki pages, ensuring consistency with latest codebase.
Bobby	`option_payoff.cpp`, `option_payoff.hpp`, `lsm_pricer.cpp`, `lsm_pricer.hpp`, `test_option_payoff.cpp`	Financial knowledge informed payoff formulation, ITM conditions, and Monte Carlo simulation with antithetic variates, along mathematical analysis with financial interpretation.
	Wiki Page, Data Analysis, Report Writing & Structure, Project Structuring & Management	Project-organization skills supported documentation, report structure, and progress monitoring.
Aradhana	`underlying_sde.cpp`, `underlying_sde.hpp`, `lsm_pricer.cpp`, `lsm_pricer.hpp`, `test_lsm_pricer.cpp`, `test_underlying_sde.cpp`	Mathematical modelling strength supported GBM, jump-diffusion, path simulation, and backward induction implementation.
	Wiki page, Report Writing	Report-writing contributions maintained clarity and focus throughout.
Joanna	`bs_pricer.cpp`, `bs_pricer.hpp`, `convergence_analyser.cpp`, `convergence_analyser.hpp`, `generate_convergence_output.hpp`, `Convergence.ipynb`, `test_bs_pricer.cpp`, `test_convergence_analyser.cpp`	Experience with large codebases supported testing, debugging, and convergence analysis. A detail-oriented approach helped identify unclear outputs and tighten project scope.
	Report Writing & Wiki Page	Report-writing contributions, focused on conciseness and clarity.

2 Methodology

The section summarises the project structure by describing the modelling approach, Object Oriented Programming (OOP) structure, algorithm design, and memory management decisions.

2.1 Theoretical Modelling Approach

The project prices American put options under risk-neutral dynamics. As a benchmark specification, the underlying asset follows GBM,

$$dS_t = rS_t\,dt + \sigma S_t\,dW_t,$$

where LSM algorithm can be compared with a finite-difference method in the standard one-factor setting. The stochastic process abstraction admits alternative specifications, including jump-diffusion dynamics, reflecting the broader class of problems suited to simulation-based methods. [1]

In the LSM method, N asset-price paths are generated on a discrete exercise grid via `simulatePaths`. At maturity, each path value is given by the terminal payoff. The price is then computed by backward induction, implemented in `backwardInduction`. At each exercise date t_m, using only In-The-Money (ITM) paths, the continuation value is approximated by the conditional expectation

$$C(S_{i,m}) = \mathbb{E}[Y_i \mid S_{i,m}], \qquad Y_i = e^{-r\Delta t}V_{i,m+1},$$

where $V_{i,m+1}$ denotes the next-step value on path i. Following the literature [1], this expectation is estimated by least-squares regression onto a finite set of basis functions, including monomials and Laguerre polynomials. The corresponding design matrix and response vector:

$$X = \begin{pmatrix} \phi_1(S_{1,m}) & \phi_2(S_{1,m}) & \cdots & \phi_K(S_{1,m}) \\ \phi_1(S_{2,m}) & \phi_2(S_{2,m}) & \cdots & \phi_K(S_{2,m}) \\ \vdots & \vdots & \ddots & \vdots \\ \phi_1(S_{n,m}) & \phi_2(S_{n,m}) & \cdots & \phi_K(S_{n,m}) \end{pmatrix}, \qquad Y = \begin{pmatrix} Y_1 \\ Y_2 \\ \vdots \\ Y_n \end{pmatrix}.$$

The least-squares problem is solved in `ols_regression`, via column-pivoted QR decomposition, avoiding direct inversion of $X^\top X$, which risks numerical instability as ITM filtering reduces the path subset remaining stable under rank-deficiency.

$$\hat{\beta} = \arg\min_{\beta} ||X\beta - Y||_2, \qquad \hat{C}_i = X_i\hat{\beta}$$

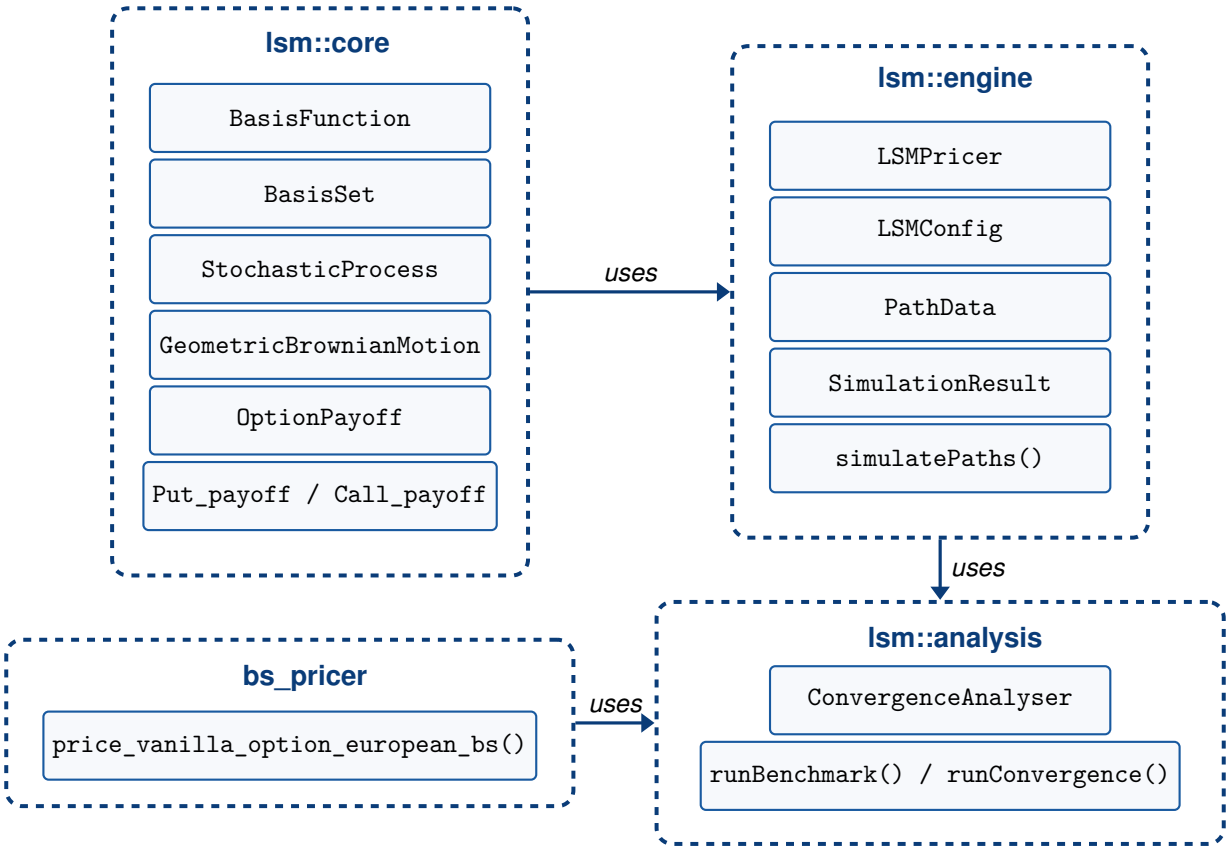
These fitted continuation values $\hat{C}_i$ are compared with immediate exercise payoffs to determine the optimal stopping decision. The resulting pathwise cashflows are discounted to time 0 and averaged to obtain the option price. [1]

To benchmark the implementation, a finite-difference method is employed in the Black–Scholes setting. The pricing PDE is solved backwards on a discretised grid, with the American constraint enforced at each step through

$$V_j = \max\{V_j, \text{payoff}(S_j)\}.$$

This provides a deterministic reference for assessing the accuracy of the LSM implementation.

2.2 Modular Structure



The codebase is organized into a modular namespace structure:

- `lsm::core` contains the fundamental abstractions.
- `lsm::engine` contains the pricing infrastructure.
- `lsm::analysis` contains the `ConvergenceAnalyser`, coordinating benchmarking experiments across alternative parameter settings.

This separation improves clarity and extensibility while avoiding name collisions. The pricing engine depends only on abstract `lsm::core` interfaces, allowing implementations to vary without modifying the core algorithm.

2.3 Object-Oriented Programming Structure

The pricing engine uses interchangeable OOP components:

- Stochastic dynamics
- Payoff structures
- Regression basis
- Pricing logic

As shown in Figure 2.1, this design separates model specification from pricing logic, allowing the LSM algorithm [1] to operate independently of the chosen process, payoff type, or basis family.

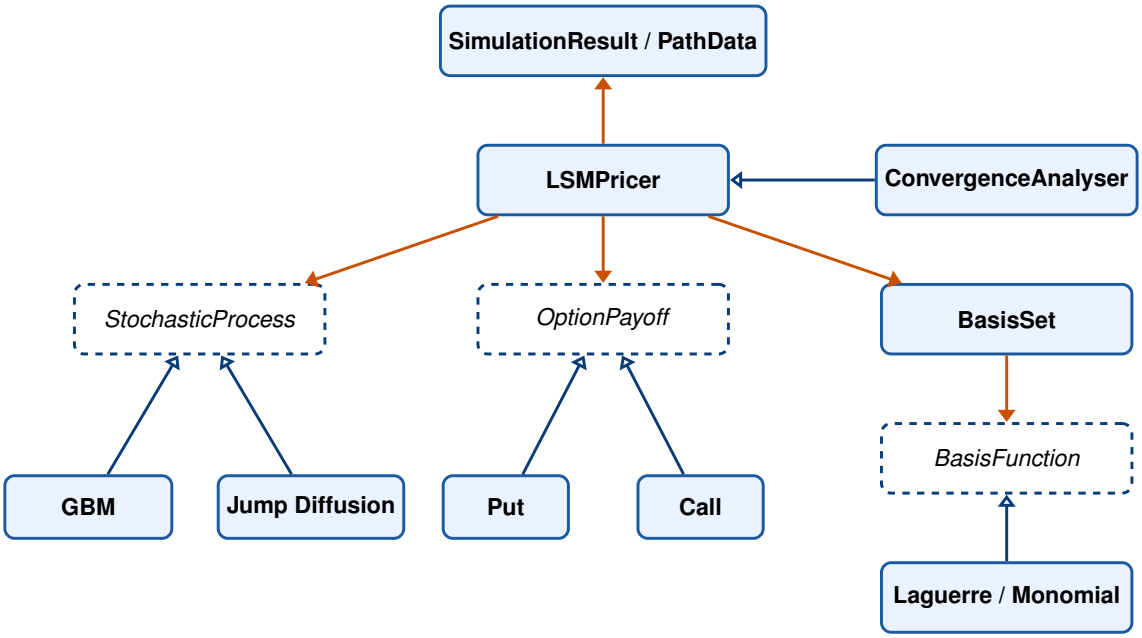


Figure 2.1: *OOP architecture of the LSM pricing framework. Dashed nodes denote abstract interfaces, solid nodes denote derived classes or supporting modules, blue arrows denote inheritance relationships, and orange arrows denote usage or ownership within the pricing engine.*

- **Abstraction:** Base classes `StochasticProcess`, `OptionPayoff`, and `BasisFunction`, define common interfaces for path evolution, payoff evaluation, and basis construction.
- **Inheritance:** Derived subclasses `GeometricBrownianMotion`, `JumpDiffusionProcess`, `Put_payoff`, `Call_payoff`, `MonomialBasis`, and `LaguerrePolynomial` **inherit** from the abstract interfaces and supply specific implementations.
- **Polymorphism:** `LSMPricer` and `ols_regression` operate on base-class references, allowing the same logic to handle alternative models, payoff and basis functions.
- **Encapsulation:** `OptionPayoff` unifies strike and payoff evaluation, `underlyingSDE` controls parameter access, `ols_regression` abstracts matrix construction, ITM filtering, and coefficient estimation, and `LSMPricer` centralises simulation, backward induction, and valuation behind a single public interface.

`LSMPricer` coordinates the workflow. `simulatePaths` generates asset trajectories from the chosen `StochasticProcess`, `backwardInduction` evaluates exercise decisions using `OptionPayoff`, and `ols_regression` estimates continuation values from `BasisSet`. Outputs are organised in `PathData` and `SimulationResult`. `ConvergenceAnalyser` extends this framework for benchmarking and sensitivity analysis, while constructing payoff and basis objects polymorphically to demonstrate the engine's reusability across configurations.

Note: *The appendix [**Section C**] contains high-level explanations of how the OOP principles have been implemented across classes.*

Layer 1 • Abstract base classes

Pure virtual interfaces — define contracts for all derived types

LAYER 1

BasisFunction «abstract»

PURE VIRTUAL
=0 double evaluate(double x) const
=0 string name() const

Strategy pattern for regression basis

StochasticProcess «abstract»

PURE VIRTUAL
=0 double stepWithNormal(s, dt, z, rng) const
NON-VIRTUAL (TEMPLATE METHOD)
+ double step(s, dt, RNG&) const
+ vec<double> simulatePath(s0, T, n, RNG&)

step() calls stepWithNormal() — template method pattern

OptionPayoff «abstract»

PROTECTED
- double K
PURE VIRTUAL
=0 double payoff(double S) const
=0 bool InTheMoney(double S) const
=0 string name() const
NON-VIRTUAL
+ double strike() const

Policy: evaluation, ITM path filter & strike access

Legend
Basis functions · Stochastic processes · Payoffs · Engines & analysis · Utilities & data · inherits · depends/uses

Layer 2 • Concrete derived classes

Override pure virtual methods; implement specific model behaviour

LAYER 2

ConstantBasis «derived»

+ double evaluate(double x)
+ string name() const

Returns 1.0 — intercept term

MonomialBasis «derived»

- int power_
+ MonomialBasis(int power)
+ double evaluate(double x)

Returns $x,\ x^2,\ x^3, \ldots$

LaguerrePolynomial «derived»

- int order_
+ LaguerrePolynomial(int n)
+ double evaluate(double x)

$L_n(x) = e^{-x/2} \cdot P_n(x);\ n \in [0, 5]$

GeometricBrownianMotion «derived»

- double r_, sigma_
+ GBM(double r, double sigma)
+ double stepWithNormal(s,dt,z)
+ double r() const; sigma() const

$dS = rS\,dt + \sigma S\,dW$ *final*

JumpDiffusionProcess «derived»

- double r_, sigma_, lambda_
+ JDP(r, sigma, lambda)
+ double stepWithNormal(s,dt,z,rng)
+ double step(s, dt, RNG&) override

Merton: $dS = (r+\lambda)S\,dt + \sigma S\,dZ$ *final*

Put_payoff «derived»

+ Put_payoff(double K)
+ double payoff(double S) const
+ bool InTheMoney(double S) const

$\max(K-S, 0)$; *ITM if* $S < K$

Call_payoff «derived»

+ Call_payoff(double K)
+ double payoff(double S) const
+ bool InTheMoney(double S) const

$\max(S-K, 0)$; *ITM if* $S > K$

uses

Layer 3 • Support & utility

RNG, BasisSet, OLS regression (free functions), config, data containers, BS/FD pricers

LAYER 3

RNG

- mt19937_64 engine
- normal_dist norm_dist
- uniform_dist uniform_dist
+ RNG(unsigned seed)
+ double normal()
+ double uniform01()

Encapsulates PRNG; seed-reproducible

BasisSet

+ vec<unique_ptr<BF>> basis
+ vec<BF*> basisPtrs() const
+ void makeLaguerreSet(int n)
+ void makeMonomialSet(int n)

Owns basis functions via unique_ptr. Factory helpers.

OLS (free functions — lsm::engine)

+ MatrixXd buildDesignMatrix(S_t, itm, basis, strike)
+ vec<double> buildYVector(cfs, itm, df)
+ vec<bool> getITMVector(S_t, payoff)
+ vec<double> Ols_regression(paths,t,cfs,itm

QR decomp via Eigen. ITM filter → design matrix → solve → continuation coeffs.

LSMConfig struct

+ int numPaths = 100 000
+ bool useAntithetic = true
+ int numExerciseDates = 50
+ double maturity = 1.0
+ double riskFreeRate = 0.06
+ unsigned rngSeed = 24

Plain struct; passed by const&

PathData struct

+ vec<vec<double>> paths
+ vec<vec<double>> cashFlows
+ int numPaths
+ int numTimeSteps

paths[i][t]: spot at t. cashFlows[i][t]: one nonzero/path.

SimulationResult struct

+ double optionValue
+ double standardError
+ double europeanValue
+ double earlyExercisePremium
+ int numPaths
+ int numExerciseDates

Output of LSMPricer::price()

BSPricer bs_pricer

+ double price_vanilla_option_euro(S0,r,sigma,K,T,call)

Closed-form Black-Scholes European price

FDPricer lsm::fd

- GBM& gbm_; OptionPayoff& payoff_
- int M_ (stockSteps); N_per_year_
+ FDPricer(gbm, payoff, stockSteps, timeStepsPerYear)
+ double price(double S0, double T) const

Crank-Nicolson / implicit FD American pricer

orchestrated by

Layer 4 • Pricing engine — LSMPricer

Main orchestrator; holds process, payoff, and basis by const reference

LAYER 4

LSMPricer

PRIVATE MEMBERS — STORED BY CONST REFERENCE OR VALUE
const StochasticProcess& process const OptionPayoff& payoff const BasisSet& basis LSMConfig config

PUBLIC
+ LSMPricer(const StochasticProcess&, const OptionPayoff&, const BasisSet&, const LSMConfig&)
+ SimulationResult price(double S0)
+ pair<SimulationResult,PathData> priceWithData(double S0)

PRIVATE — ALGORITHM STEPS
- vec<double> backwardInduction(PathData&) const
- SimulationResult computeOptionValue(vec<double>&, double, int, int) const

Longstaff-Schwartz algorithm: price() → simulatePaths() [free fn] → backwardInduction() → Ols_regression() → computeOptionValue()

analyzed by

Layer 5 • Analysis & convergence tools

Constructs independent LSMPricer per config variant via factories; writes CSV results

LAYER 5

ConvergenceAnalyser lsm::analysis

PRIVATE
- double S0, r, sigma, K, T
- function sdeFactory (double r, double sigma) → unique_ptr<StochasticProcess>
- OptionPayoff& payoffType; BasisSet& basisType
- function basisFactory (BasisSet&, int order) → void
- int order; fixedPathCount; fixedNumDates

PUBLIC
+ ConvergenceAnalyser(S0, r, sigma, K, T, sdeFactory, payoff, basis, basisFactory, order, fixedPathCount, fixedNumDates)
+ double getBSPrice(); getFDPrice()
+ double getLSMPrice(seed, numExerciseDates, order, numPaths)
+ SimulationResult getLSMResult(seed, numExerciseDates, order, numPaths, antithetic)
+ void runBenchmark(); runSeedStability()
+ void runPathConvergence(pathCounts); runDatesConvergence(exerciseDatesList)
+ void runOrderConvergence(orders); runFDConvergence(timeCounts, stockSteps)

Constructs LSMPricer per config variant via factories. Benchmarks against BS European and FD American. Writes CSV output to csv_output/.

2.4 Memory Management and Optimization

Memory management was designed around the computational demands of Monte Carlo pricing, where large path arrays and repeated regressions make allocation costs material. `StochasticProcess` in `ConvergenceAnalyser` and `BasisSet` are owned via `std::unique_ptr`, enforcing RAII and explicit ownership. Numerical data are stored in `std::vector` and Eigen structures [3], suited to dynamic path storage and linear algebra, while temporary raw pointers are used only for non-owning access. Pre-allocation and reusable buffers reduce avoidable overhead.

3 Testing

Unit tests were used to verify correctness in isolation. Integration tests were concentrated on `LSMPricer`, since simulation, payoff evaluation, regression and stopping logic interact. The convergence analyser was tested functionally, focusing on whether benchmarking helpers and CSV-generated workflows execute correctly. Examples from the test suite include:

- **Basis functions:** Closed-form correctness, polymorphic validation, basis-set structure, and invalid orders.
- **Option payoff:** Exact payoff values and ITM logic for put and call contracts.
- **Underlying SDE:** One-step dynamics, admissibility checks, and deterministic limiting cases.
- **LSM pricer:** Structural consistency, financial properties, benchmark agreement, and the literature toy example to validate backward induction directly.
- **Black-Scholes pricer:** Verified against a known At-The-Money (ATM) price and tested for put-call parity and monotonicity in spot and volatility.

3.1 Test Coverage

Coverage script `run_coverage.sh` served as a guideline for identifying any major gaps. The suite achieved 99.1% line coverage and 94.1% function coverage. The missed lines in `basis_functions`, `option_payoff`, and `underlying_sde` are mainly string-output branches used for naming or CSV labels, so additional tests for them would be of limited value. Numerically substantive parts of codebase, including regression, simulation, and backward induction, were fully exercised.

Table 3.1: *Coverage report generated by* `run_coverage.sh`

File	Lines	Exec	Cover	Missing
src/basis_functions.cpp	64	62	96%	34, 65
src/basis_functions.hpp	2	2	100%	—
src/bs_pricer.cpp	13	13	100%	—
src/convergence_analyser.cpp	238	238	100%	—
src/convergence_analyser.hpp	0	0	–%	—
src/finite_difference.cpp	58	58	100%	—
src/lsm_pricer.cpp	139	139	100%	—
src/lsm_pricer.hpp	17	17	100%	—
src/mc_paths.cpp	7	7	100%	—
src/ols_regression.cpp	52	52	100%	—
src/option_payoff.cpp	13	12	92%	55
src/option_payoff.hpp	2	2	100%	—
src/underlying_sde.cpp	64	61	95%	114, 125, 173
src/underlying_sde.hpp	10	10	100%	—
TOTAL	**679**	**673**	**99%**	—

Lines: 99.1% (673/679) *Functions:* 94.1% (80/85) *Branches:* 56.3% (695/1234)

4 LSM Pricer Visual Analysis for Put Option

The following results provide an analysis of the continuation curve and simulated paths for **put option**, strike price $K = 1.1$ and time t.

4.1 Simulated Paths: ITM & OTM and Decision Map

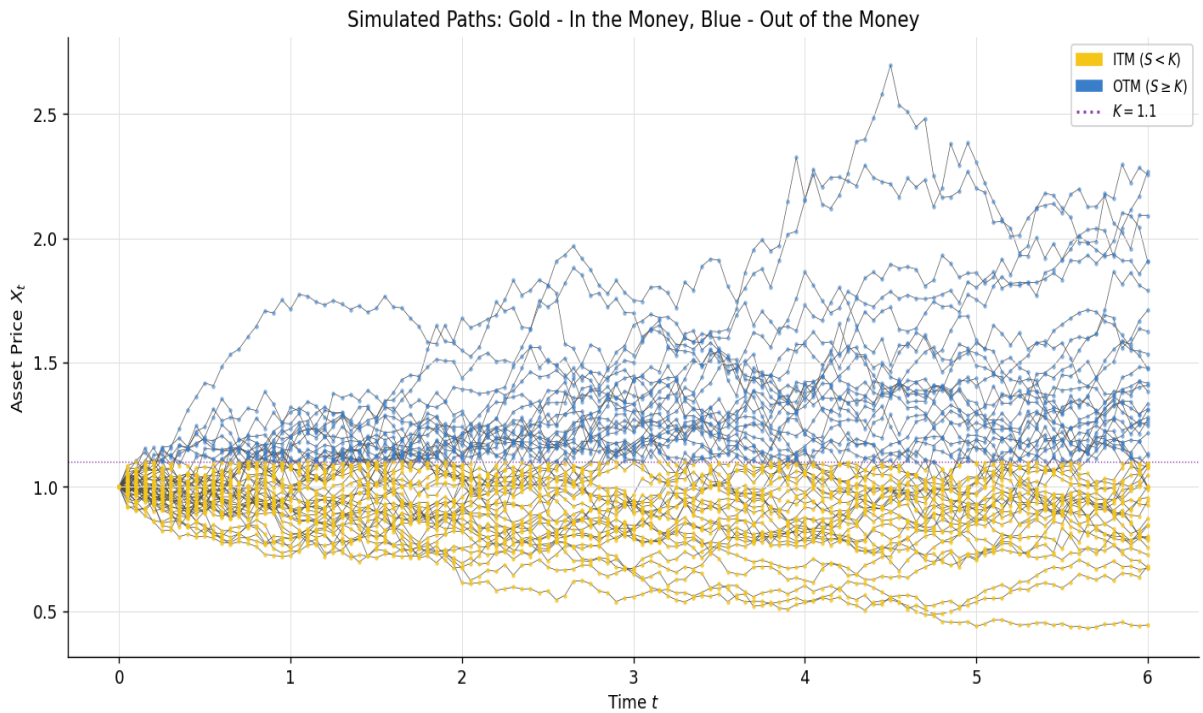


Figure 4.1: *In the Money GBM paths*

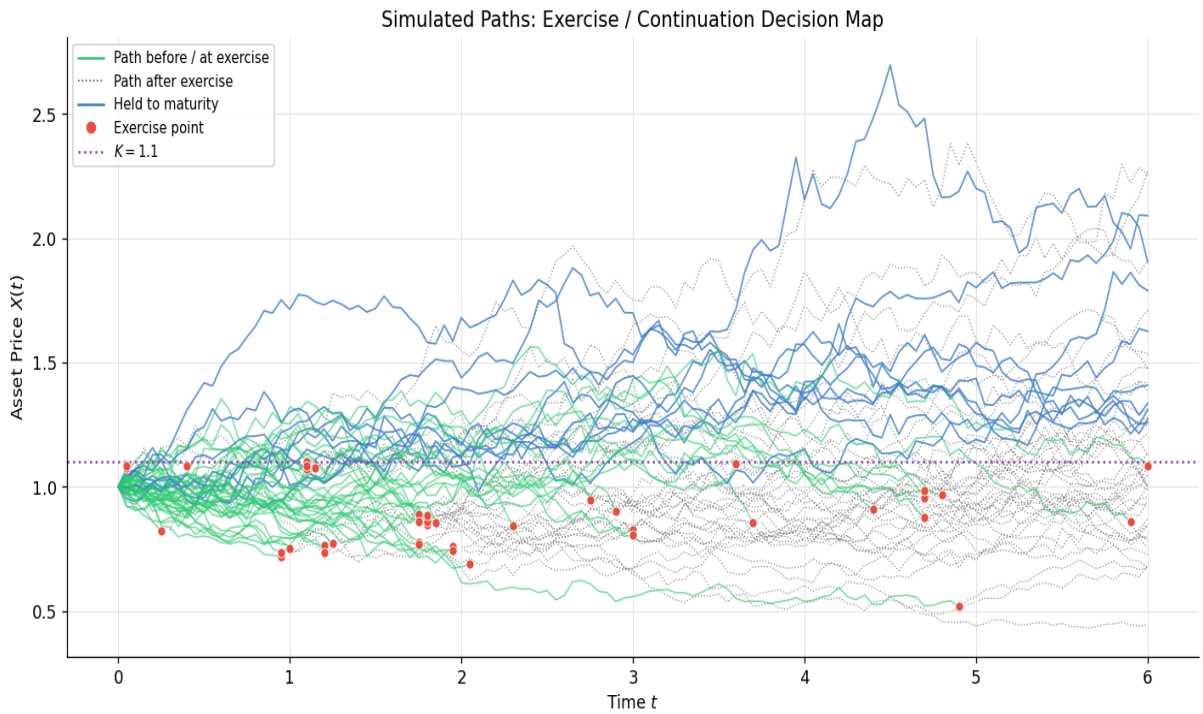


Figure 4.2: *Decision Map of Simulated Paths*

Analysis:

1. Figure 4.1 represents simulated GBM asset paths. Gold segments denote ITM states and blue segments denote Out-Of-The-Money (OTM) states. The paths spread over time, reflecting increasing uncertainty. By maturity, paths above K imply zero payoff, while paths below K lie in the exercise-relevant region. This highlights both the uncertainty driving the continuation decision and the range of downside outcomes that a risk manager should consider.

2. Figure 4.2 displays the simulated exercise/continuation map. Exercise points occur predominantly when the asset price is at or below K, implying ATM or ITM where immediate exercise becomes preferable to continuation. Paths above K are mostly continued, since the put has little or no intrinsic value. This illustrates the pathwise stopping rule produced by the LSM algorithm and from a risk-management perspective, the cost of delaying exercise when future price movements may erode option value.

4.2 LSM Regression: Continuation Value

Analysis:

- The LSM regression step estimates continuation value at t_{n-1}. It shows discounted simulated cashflows and the fitted regression curve used to approximate continuation; the core numerical step of the LSM method. It compares exercise and continuation values, identifying the optimal stopping region and, from a risk-management perspective, when early exercise is preferable to continued exposure.

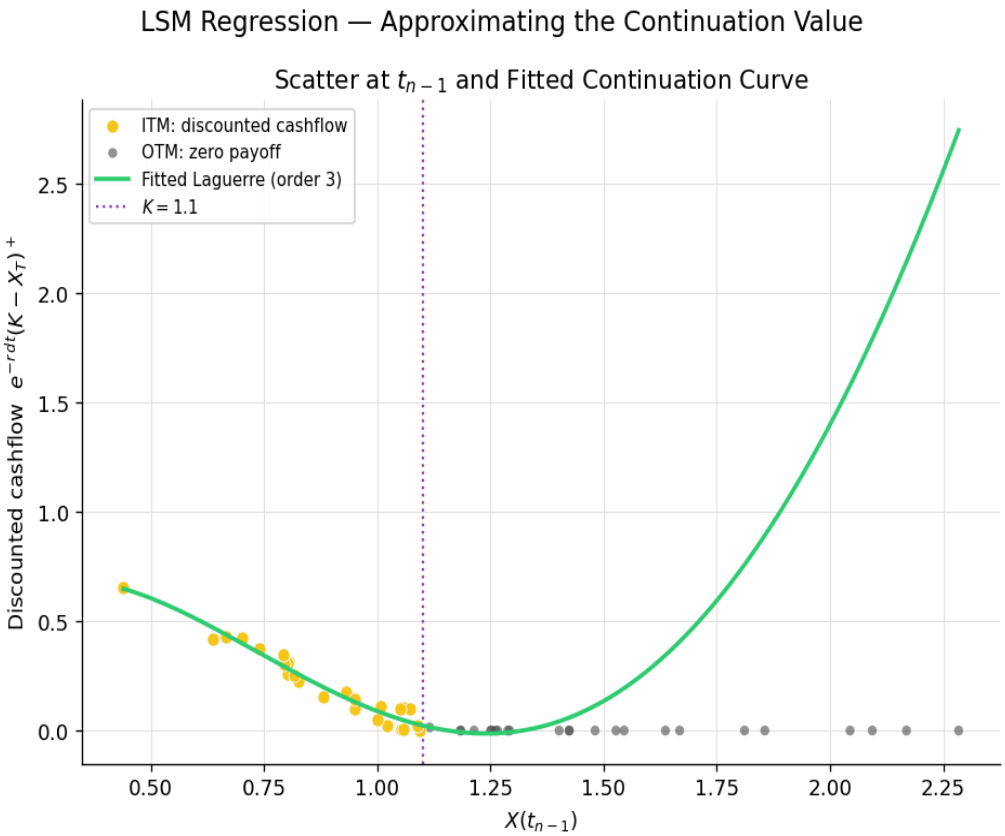


Figure 4.3: *Fitted Continuation Curve*

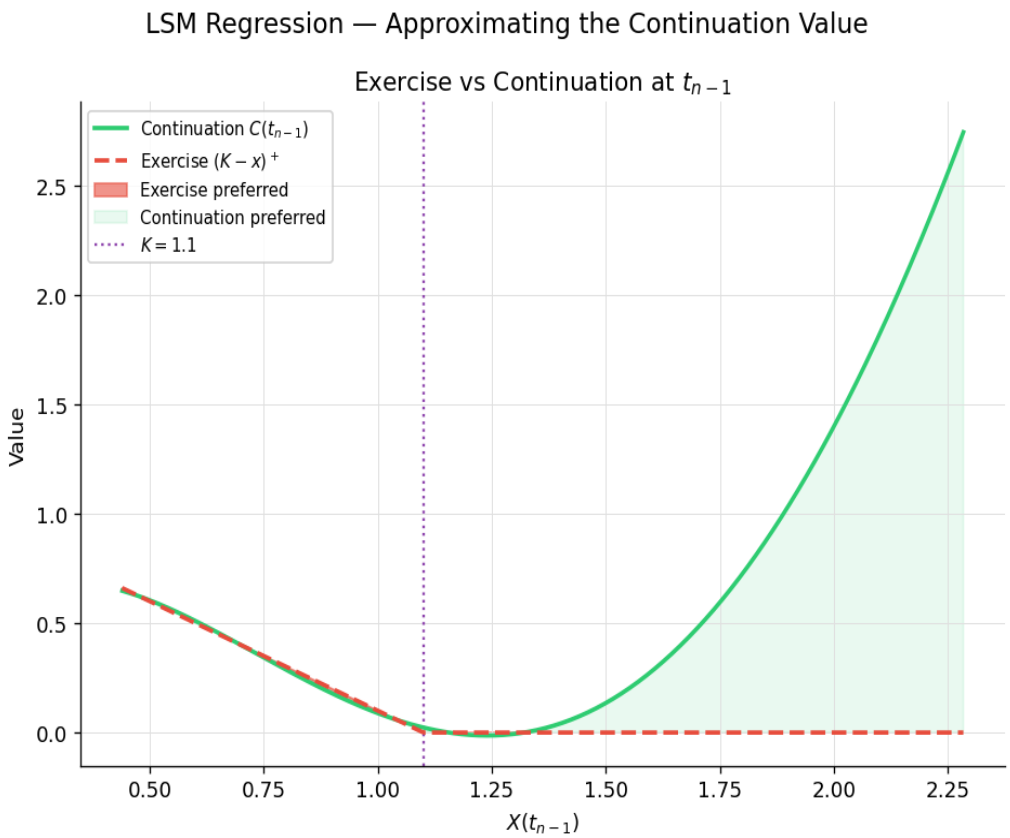


Figure 4.4: *Exercise Continuation Curve*

5 Benchmark and Convergence Results

5.1 Validation

Table 5.1: *American put prices from BS (European), FD, and LSM pricers across 12 parameter combinations ($K = 40$, $r = 0.06$). Early exercise premium = LSM − BS.*

S_0	σ	T	BS (European)	FD (American)	LSM (American)	Early Ex. Premium
36	0.2	1	3.844	4.486	4.467	0.623
36	0.2	2	3.763	4.848	4.835	1.072
36	0.4	1	6.711	7.109	7.073	0.361
36	0.4	2	7.700	8.514	8.424	0.724
40	0.2	1	2.066	2.319	2.291	0.224
40	0.2	2	2.356	2.890	2.838	0.482
40	0.4	1	5.060	5.318	5.308	0.248
40	0.4	2	6.326	6.923	6.881	0.555
44	0.2	1	1.017	1.113	1.114	0.097
44	0.2	2	1.429	1.693	1.696	0.267
44	0.4	1	3.783	3.953	3.923	0.140
44	0.4	2	5.202	5.647	5.622	0.420

Validation was based on the 12 American put configurations reported in the literature [1]. Across all cases, LSM prices are close to the finite-difference benchmarks, and the early exercise premium relative to the Black–Scholes European price remains strictly positive. LSM prices were consistently below the FD values, consistent with Proposition 1 of Longstaff and Schwartz: for any finite basis and exercise grid, the LSM stopping rule yields a lower-bound approximation to the true American option value. These results support both the numerical accuracy and financial consistency of the implementation.

5.2 Convergence Analysis

The LSM algorithm's accuracy depends on three key parameters:

- Number of paths (N)
- Exercise dates (M)
- Polynomial basis order (p)

Path Convergence (Figure 5.1): The empirically fitted convergence rate of $O(N^{-0.64})$ is consistent with theoretical Monte Carlo rate of $O(N^{-0.5})$ [1] with small excess due to noise, as we only used 1 seed. The LSM price stabilizes around $N = 10{,}000$ paths, with absolute error falling below 0.01 for $N \geq 8{,}000$, indicating a practical accuracy-efficiency trade-off.

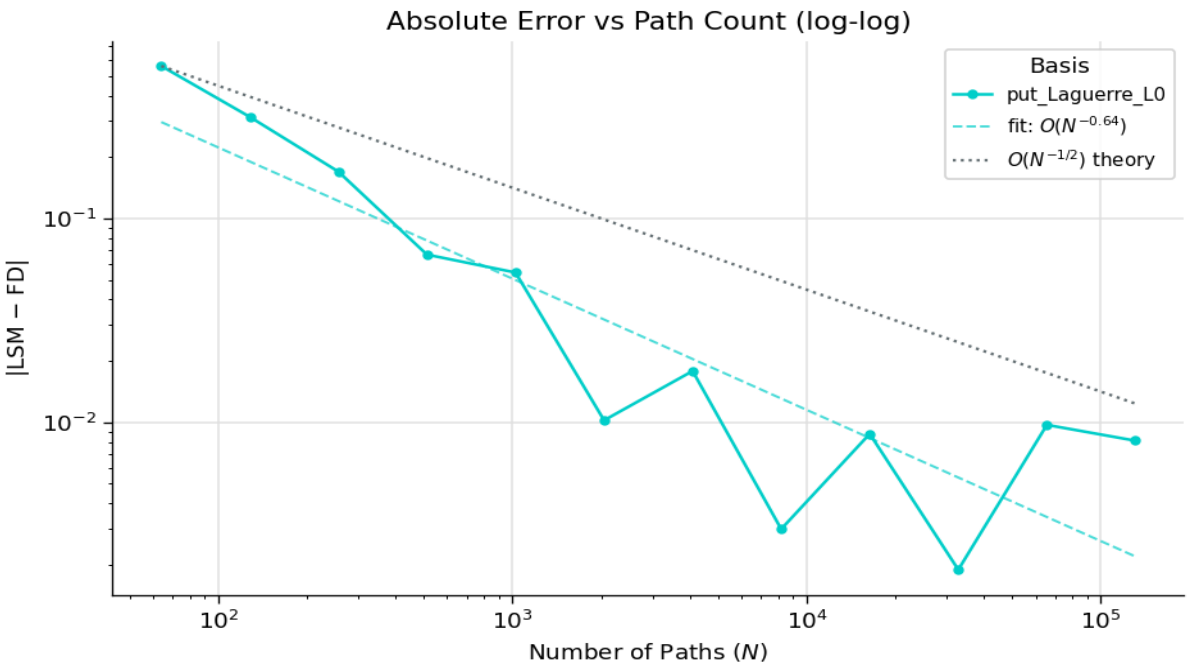


Figure 5.1: *Absolute error $|LSM - FD|$ vs number of simulated paths N on a log-log scale.*

Exercise Dates Convergence (Figure 5.2): M controls how closely the algorithm approximates continuous early exercise. The convergence rate of $O(M^{-0.99})$ is consistent with the theoretical $O(M^{-1})$ rate for time discretisation [1]. At $M = 1$, LSM price reduces to European value, providing additional sanity check on the implementation. The price stabilizes beyond $M \approx 50$, beyond which increasing M shows negligible improvement.

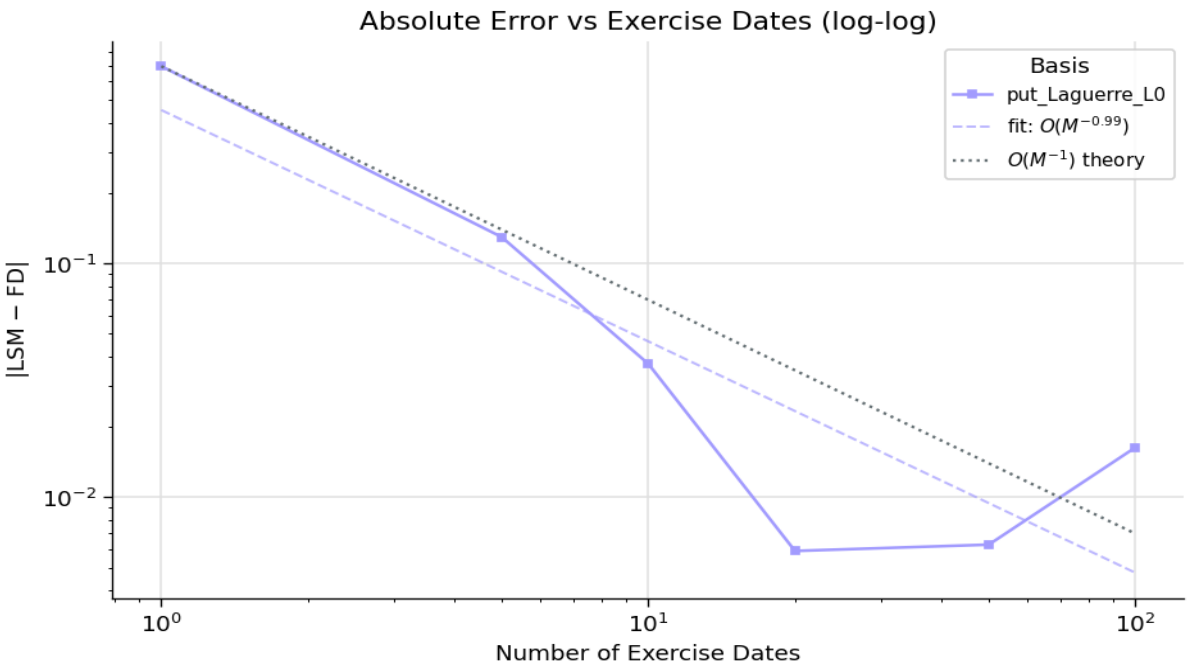


Figure 5.2: *Absolute error $|LSM - FD|$ vs number of exercise dates M on a log-log scale.*

Basis Order Convergence (Figure 5.3): Substantial accuracy gains are observed from $p = 1$ to $p = 3$, with errors falling from 0.054 at $p = 1$, to 0.022 at $p = 2$, and 0.006 at $p = 3$. Beyond this, $p = 4$ yields marginal improvement, and $p = 5$ shows a slight increase in error, due to overfitting or numerical instability in the regression. This suggests that, for a standard American put, a low-order polynomial basis is sufficient, whereas higher-order terms add complexity without clear numerical benefit.

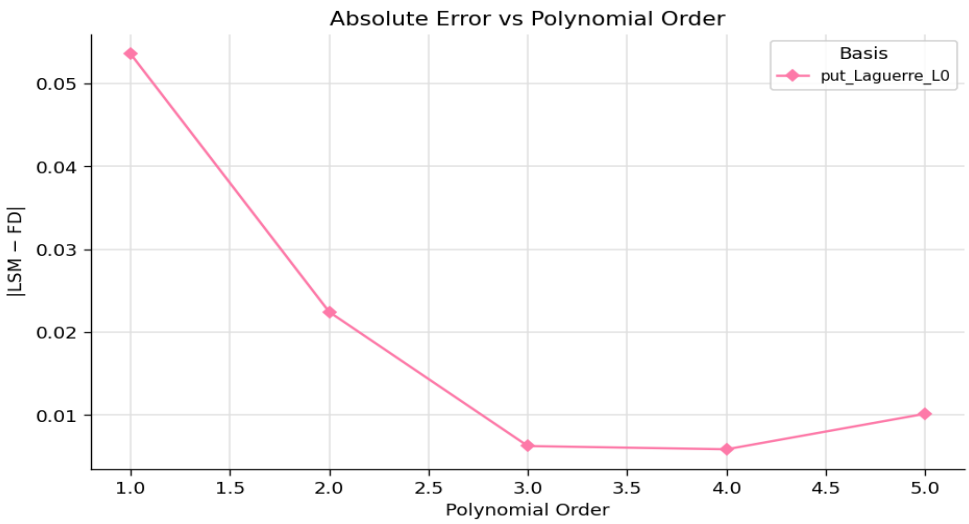


Figure 5.3: *Absolute error $|LSM - FD|$ as a function of polynomial basis order p.*

Seed Sensitivity (Figure 5.4): Running the algorithm across 10 seeds with fixed ($N = 10\,000$, $M = 50$, $p = 3$) shows the resulting prices scattered around the mean of 4.483, which lies close to the FD reference of 4.487. The inter-quartile range is narrow, with outliers at seeds 1 and 8. The average per-seed standard error is 0.030, indicating that the remaining variation is primarily Monte Carlo noise.

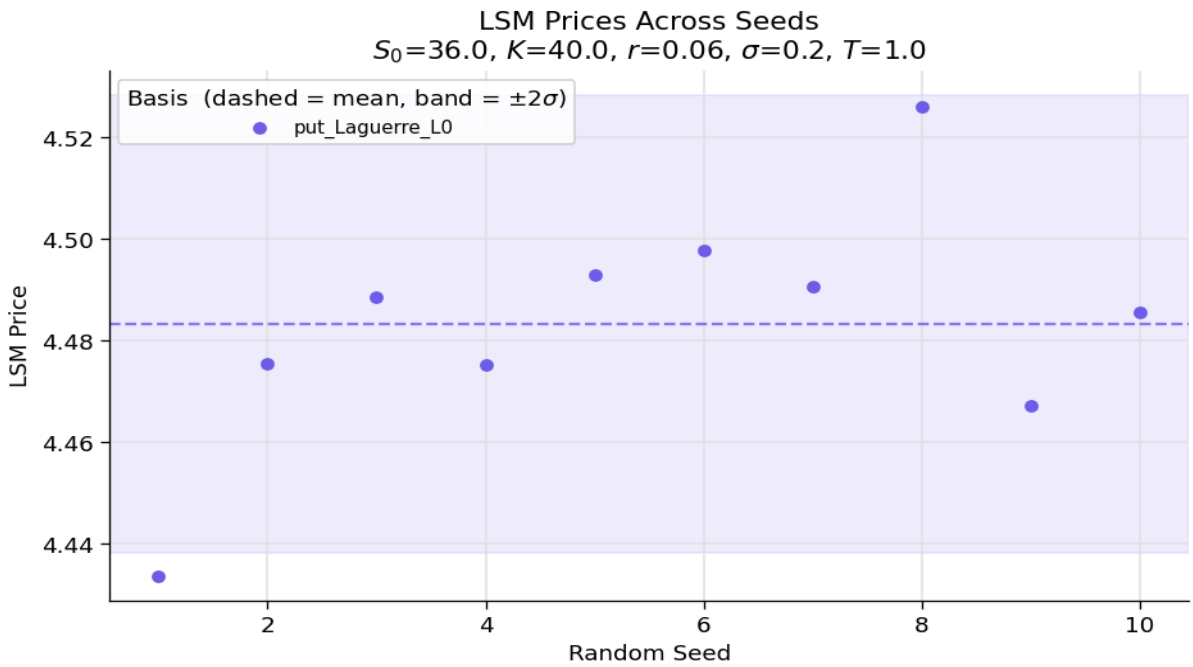


Figure 5.4: *LSM prices across 10 random seeds ($N = 10{,}000$, $M = 50$, $p = 3$). Dashed line shows the mean; shaded band shows $\pm 2\sigma$.*

Variance Reduction (Figures 5.5 5.6): Antithetic sampling reduces the average standard error from 0.0295 to 0.0191, a reduction of 35.3%, consistent with the variance reduction expected when paths are negatively correlated. Antithetic standard errors are highly consistent across seeds (all close to 0.019), whereas the plain standard errors show more variation. The mean price is unaffected (4.483 in both cases), so efficiency is improved without introducing bias.

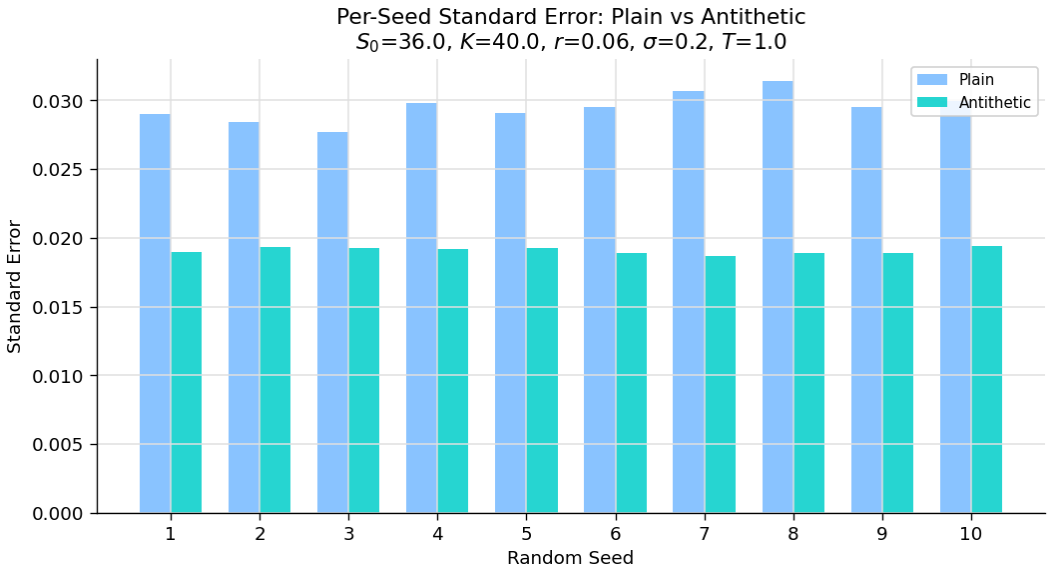


Figure 5.5: *Per-seed standard error: plain vs antithetic. Variance reduction from antithetic variates across 10 seeds.*

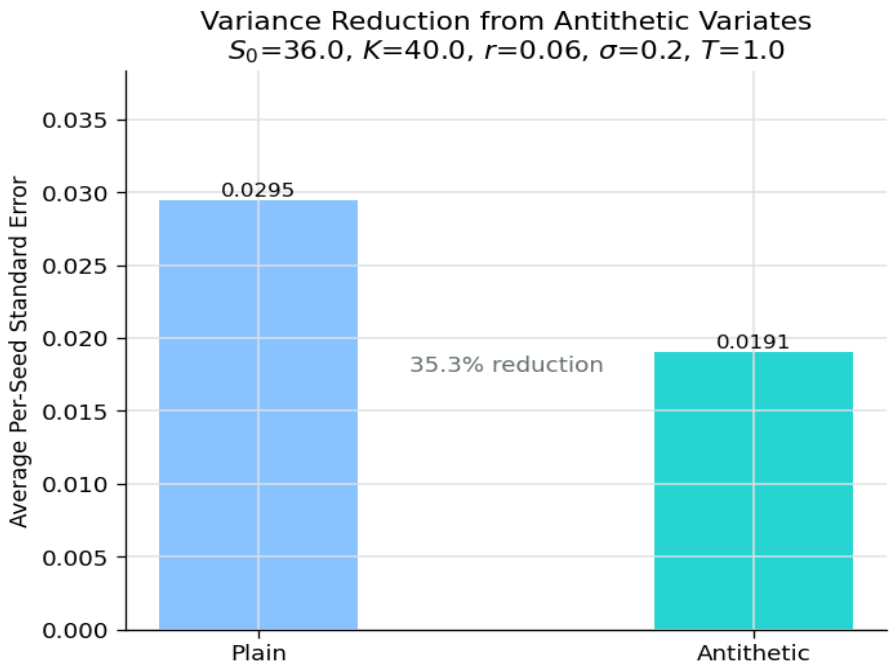


Figure 5.6: *Average standard error summary. Variance reduction from antithetic variates across 10 seeds.*

Summary The convergence analysis confirms theoretical expectations across all parameters, with optimal performance achieved at $N \geq 10,000$, $M \geq 50$, and $p = 3$, while antithetic variates provide meaningful efficiency gains.

6 Project Limitations

Main Limitation:

1. Lasso and Ridge regularization methods could also be explored, if a broader set of higher order polynomials were considered.
2. Parallel computing was not implemented, therefore only 1 seed was used for Monte Carlo simulations, which might result in correlations between randomly generated data.

Other Limitation:

1. Near-ATM regions remains numerically challenging since small price changes can alter the stopping decision, leading to slow convergence and high variance.

6.1 Further Scopes

1. Different types of financial instruments (swaps, Asian options, bonds) can be implemented through the same abstract class structure utilized in `OptionPayoff`.
2. Parallel computation implementation of LSM pricer algorithm, where time essence is the key factor for analysis.
3. LSM application to calculate risk capital for life insurance contracts possessing embedded early-exercise, shares feature of early exercise in American options [4].
4. Implementing Neural Networks instead of OLS regression can approximate complex continuous functions, capturing non-linearities in continuation value that OLS misses.

Conclusion

Following Longstaff & Schwartz [1], this project developed an OOP implementation of the LSM algorithm, leveraging C++ features and modular architecture for performance. The results demonstrate sound methodology, effective code structures, and financially consistent performance, supported by benchmarking, convergence analysis, and rigorous testing. We have dedicated every member's strengths to achieving our aims and documenting the progress in the report.

References

1. Longstaff, F. A. & Schwartz, E. S. Valuing American Options by Simulation: A Simple Least-Squares Approach. *Review of Financial Studies* **14,** 113–147. doi:10.1093/rfs/14.1.113 (2001).
2. Marsceill, P. & contributors. *Just the Docs* https://just-the-docs.com. Jekyll theme for documentation sites, MIT License. 2017–2026.
3. Eigen Project. *The Matrix Class - Eigen Documentation* https://libeigen.gitlab.io/eigen/docs-5.0/group__TutorialMatrixClass.html. Accessed: 29 March 2026. 2026.
4. Bacinello, A. R., Millossovich, P., Olivieri, A. & Pitacco, E. Pricing life insurance contracts with early exercise features. *Journal of Computational and Applied Mathematics* **233,** 27–35. doi:10.1016/j.cam.2008.05.036 (2009).
5. cppreference.com. *C++ Standard Library* https://en.cppreference.com/w/cpp/standard_library.html. Accessed: 29 March 2026. 2026.
6. Kitware, Inc. *cmake-buildsystem(7)* Accessed: 03 April 2026. 2024. https://cmake.org/cmake/help/latest/manual/cmake-buildsystem.7.html.

Appendix

A Additional Task Allocation Details

	Tasks	Work details
Srikar	`basis_functions.cpp`, `basis_functions.hpp`	Implemented the complete Basis Function part of the LSM Pricer.
	`lsm_pricer.cpp`, `lsm_pricer.hpp`	Developed the `Price` and `PriceWithData` classes of the LSM pricer.
	`main.cpp`	Constructed complete `main.cpp` for appropriate data output, needed for data analysis and plots.
	`Analysis_plots.ipynb`	Developed a Python program for analysis and fitting plots using output data from `main.cpp`
	`CMakeLists.txt`	Optimised the CMakeLists of main project and tests to reduce the build and compiling times.
	Wiki page	Maintained documentation of the basis function and main program, later migrated to GitHub Pages.
	GitHub Pages	Responsible for the GitHub Pages and deployed pages using information from the repository's wiki.
	Git and Repository Management	Managed Git commits, merge conflicts, branch issues and divergence. Fixed autograding errors throughout the project.
	Project Structuring	Set up the class structure and modular structure of the project. As well as debugging all the project files and guiding team members on the appropriate fixes and methods to align with the original methodology. Fixed the external library errors and any other project concerned with individual teammate.
	Report Writing	Written the report concerning the assigned tasks, tikz figures/flowcharts of class structures of all other project files, appendix and other parts of the report as a whole.
Alex	`ols_regression.cpp`, `ols_regression.hpp`, `test_OLS_regression.cpp`	Implemented the OLS machinery using Eigen's QR decomposition solve function with helper functions for constructing the design matrix and continuation values. Added testing consistent with examples found in the paper to ensure correctness.
	`finite_difference.cpp`, `finite_difference.hpp`, `test_finite_difference.cpp`	Used all the available information in the paper to find a finite difference scheme with best fit to the paper. Added tests that ensure it fits the numerical results in the paper with the same parameters.

	Tasks	Work details
	Wiki page	Added method for OLS regression with in-depth mathematical explanation on the advantages of QR decomposition for stability of results in OLS regression. Included brief explanation of path simulation function that feeds the data to the OLS regression. Ensured the final version of the Github pages were inline with the revised codebase.
Bobby	`option_payoff.cpp`, `option_payoff.hpp`	Implemented the Option Payoff function for use in `ols_regression` and `lsm_pricer`. As well as providing explanations on the codes and logic behind it.
	`lsm_pricer.cpp`, `lsm_pricer.hpp`	Focused on `simulatePaths` implementation through using Monte Carlo simulation and the Antithetic Variates.
	`test_option_payoff.cpp`	Created and included test cases for the Option Payoff to ensure consistent performance for the payoff functions and ITM conditions.
	Wiki Page	Written a wiki page on Option Payoff and testing to document important aspects of the code and include in the report.
	Data Analysis	Provided written analysis in the report for the simulation plots generated in Python, linking to investment decisions and strategy.
	Report Structure Management	Ensured the report has a consistent written structure, presentability, proofreading, and flow throughout the report.
	Report Writing	Written the report for the personal allocated tasks, and contributed to writing in other areas of the report to ensure the overall flow is consistent.
	Project Structuring & Management	Provided clear documentation for the group to outline the workload and track the group project progress.
Aradhana	`underlying_sde.cpp`, `underlying_sde.hpp`	Implemented the stochstic process framework, including the random number generator, the abstract `StochasticProcess` interface, the Geometric Brownian Motion model, and the Jump Diffusion model.
	`lsm_pricer.cpp`, `lsm_pricer.hpp`	Implemented parts of `LSMPricer` class, namely defined the class structure, `backwardInduction`, `computeOptionValue` methods.
	`test_lsm_pricer.cpp`, `test_underlying_sde.cpp`	Designed and implemented rigorous tests for the LSM pricer and underlying SDE modules, covering mathematical correctness, edge cases, structural consistency, and financial properties.

	Tasks	Work details
	Wiki page	Produced wiki documentation for the LSM-Pricer and underlying SDE modules explaining the implementation, design structure, and key methodological ideas
	Report Writing	Written the report for the personal allocated tasks and helped keeping areas concise and project-focused.
Joanna	`bs_pricer.cpp`, `bs_pricer.hpp`	Implemented the Black-Scholes European pricer used as the analytical benchmark for validation and convergence analysis.
	`convergence_analyser.cpp`, `convergence_analyser.hpp`	Designed and implemented the `ConvergenceAnalyser` class, including all convergence routines sweeping paths, exercise dates, and polynomial order, as well as seed stability analysis with antithetic variate comparison. All routines write structured CSV output for downstream Python analysis.
	`generate_convergence_output.hpp`, `Convergence.ipynb`	Implemented the helper for batch CSV generation used by the analysis notebook. Developed the Python notebook producing all convergence and variance reduction plots shown in Section 5.
	`test_bs_pricer.cpp`, `test_convergence_analyser.cpp`	Wrote comprehensive test suites for the Black-Scholes pricer and convergence analyser, covering financial properties, monotonicity, put-call parity, and structural CSV output validation. Added additional tests across modules to close coverage gaps identified by `run_coverage.sh`, contributing to the 99.6% line coverage in Table 3.1.

Table A.1: *Task Details*

B Libraries

Library	Description	Files Using It	Usage
`<algorithm>`	Standard algorithms (sorting, searching, min/max)	main.cpp	Used via `std::max`, `std::min` in payoff calculations and file operations.
`<cmath>`	Mathematical functions (exp, log, sqrt, pow, fabs)	main.cpp, basis_functions.hpp, ols_regression.hpp	`std::exp`, `std::sqrt` for financial calculations, Laguerre polynomials, and design matrices.
`<cstdlib>`	General utilities (`system`, `exit`, `rand`)	main.cpp	`system(cmd.c_str())` used to create output directories.
`<fstream>`	File input/output streams	main.cpp	Writing CSV outputs using `ofstream` via utility functions.
`<iomanip>`	Output formatting tools	main.cpp	`fixed`, `setprecision` for consistent numerical formatting.
`<iostream>`	Standard I/O streams	main.cpp	Console output for pricing results and error reporting.
`<memory>`	Smart pointers for memory management	main.cpp, lsm_pricer.hpp, basis_functions.hpp	Managing ownership of objects in stochastic processes, basis sets.
`<stdexcept>`	Standard exception handling	main.cpp, basis_functions.hpp, ols_regression.hpp	Throwing `runtime_error` for file failures and invalid inputs.
`<string>`	String handling	All source files	File paths, error messages, and configuration identifiers.
`<vector>`	Dynamic array container	All source files	Storing simulation paths, cash flows, and regression data.
`<functional>`	Function objects and utilities	ols_regression.hpp	Used indirectly for lambdas, Eigen internals.
`<random>`	Random number generation	underlying_sde.hpp	`mt19937_64`, normal and uniform distributions for Monte Carlo simulation.
`<cstddef>`	Core type definitions	underlying_sde.hpp	Use of `std::size_t` for indexing and path lengths.
`Eigen/Dense`	Linear algebra library	ols_regression.hpp	`Eigen::MatrixXd` for regression design matrices and OLS solving.

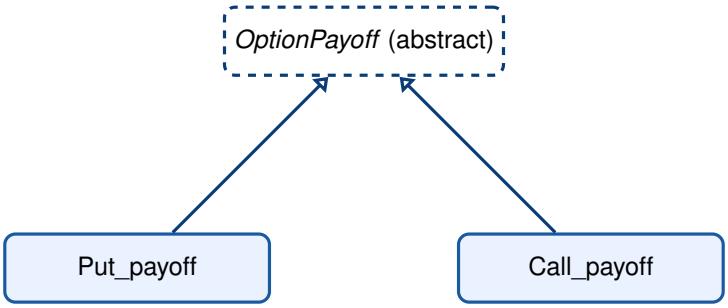
Notes: All files are C++ source/header files. The only external dependency is `Eigen`, a header-only linear algebra library. The project uses modern C++ features like smart pointers and RAII without relying on additional frameworks.

The documentation, along with the examples provided for the C++ Standard Library [5] and the Eigen library [3], was referred for correct usage throughout this project. We used the Eigen library to have highly efficient, numerically stable, and simple solving of the OLS regressor via QR decomposition.

C Object Oriented Programming Structure

In this section, we focus on the main structures: abstraction, inheritance, polymorphism, and encapsulation contained within our project.

C.1 Option Payoff



Abstraction

Represented by the abstract base class `OptionPayoff` allows the pricing engine to operate independently of the specific payoff type.

```
class OptionPayoff {
public:
    virtual double payoff(double S) const = 0;
    virtual bool InTheMoney(double S) const = 0;
};
```

Inheritance

`OptionPayoff` represent inheritance as `Put_payoff` and `Call_payoff` rely on the base class. Both `Put_payoff` and `Call_payoff` contain the strike price K and must implement the `payoff()` and `InTheMoney()`, which both share a common structure. While `OptionPayoff()` initializes K, the classes do not need to re-implement the strike.

```
OptionPayoff::OptionPayoff(double strike) : K(strike) {}
...
Put_payoff::Put_payoff(double strike) : OptionPayoff(strike) {}
Call_payoff::Call_payoff(double strike) : OptionPayoff(strike) {}
```

Polymorphism

The main part is the virtual functions,

```
virtual double payoff(double S) const = 0;
virtual bool InTheMoney(double S) const = 0;
```

`OptionPayoff` does not determine the behavior; other classes have individual versions, allowing calls through a base pointer to behave differently depending on object type. This is polymorphism, as the same function name can exhibit different behaviors depending on whether the object is a put or a call.

The ITM function represents polymorphism, meaning that having the same method for ITM gives different results depending on the object type.

[Virtual methods `OptionPayoff` and `InTheMoney()` enable runtime dispatch, allowing the same pricing logic to support payoff types.]

```
bool Put_payoff::InTheMoney(double S) const {
    return S < K;
}
bool Call_payoff::InTheMoney(double S) const {
    return S > K;
}
```

Encapsulation

Demonstrated through `OptionPayoff` class, where the strike price K and payoff logic are bundled within a single object. K is declared as `protected`, meaning external code cannot modify it directly, while derived classes `Put_payoff` and `Call_payoff` retain access through inheritance.

```
protected:
double K;
public:
double strike() const { return K; }
```

Read-only access for external code is provided through public `strike()` getter method, ensuring the internal state of the object can be observed, not altered. The payoff logic is hidden behind the abstract `payoff()` and `InTheMoney()` interfaces. The caller needs only to invoke these methods without knowing how the computation is performed internally.

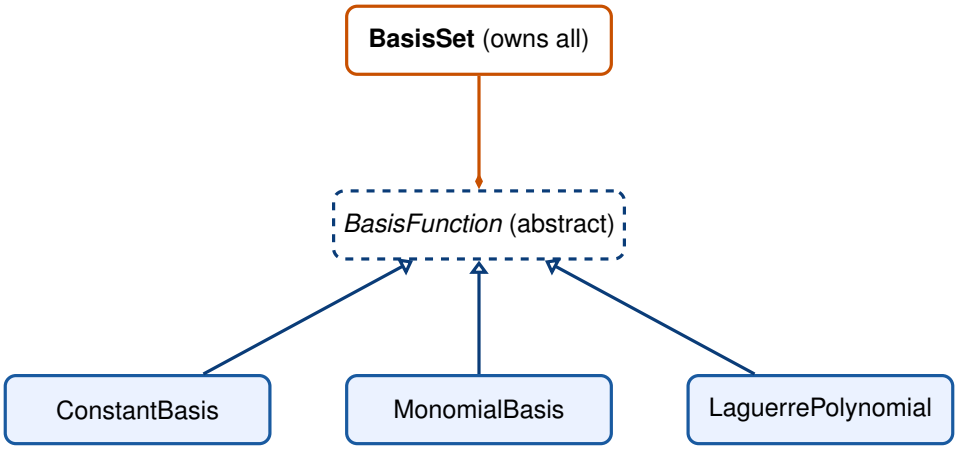
```
double Put_payoff::payoff(double S) const {
return std::max(K - S, 0.0);
}
double Call_payoff::payoff(double S) const {
return std::max(S - K, 0.0);
}
```

This means that each payoff object is self-contained, carrying the data needed and the rules for how to act on it.

[The strike is stored as a protected member and accessed through public interfaces, ensuring payoff logic and associated parameters remain self-contained.]

C.2 Basis Functions

The `basis_functions.hpp` and `basis_functions.cpp` implements regression basis functions used inside the Longstaff-Schwartz backward-induction loop. The chosen design keeps each concern isolated, so adding a new basis family requires writing one new subclass and nothing else.



Abstraction

`BasisFunction`, an abstract base class with two pure-virtual methods, defines *what* must be implemented without providing any implementation itself.

```
class BasisFunction {
public:
    virtual ~BasisFunction() = default;
    virtual double evaluate(double x) const = 0;  // no body, must override
    virtual std::string name() const = 0;
};
```

The Ordinary Least Square (OLS) regression loop calls `->evaluate()` through the base class pointer, without knowing the underlying concrete type:

```
// ols_regression.cpp - works for any BasisFunction subclass
for (int k = 0; k < K; ++k)
    X(row, k) = basis[k]->evaluate(S_t[i]);
```

[`BasisFunction` defines a common abstract interface for regression basis terms through `evalute()` and `name()`, allowing the regression engine to construct the design matrix without reference to any specific basis family.]

Inheritance

All three concrete classes inherit from `BasisFunction`, each satisfying the same structure such as,

```
class ConstantBasis : public BasisFunction {...};
class MonomialBasis : public BasisFunction {...};
class LaguerrePolynomial : public BasisFunction {...};
```

`BasisSet`'s factory methods create derived objects but store them behind the base pointer, so the rest of the system sees only `BasisFunction`:

```
void BasisSet::makeLaguerreSet(int numTerms) {
    basis_.push_back(std::make_unique<ConstantBasis>());
    for (int i = 0; i < numTerms; ++i)
```

```
        basis_.push_back(std::make_unique<LaguerrePolynomial>(i));
}
```

[ConstantBasis, MonomialBasis, and LaguerrePolynomial inherit from BasisFunction, each implementing the required interface while preserving a uniform structure for use in regression.]

Polymorphism

Implemented through a common BasisFunction base class that declares evaluate as a pure virtual method. The three concrete subclasses, ConstantBasis, MonomialBasis, and LaguerrePolynomial, each provide their own implementation, and the pricer stores them as std::vector <std::unique_ptr<BasisFunction», calling evaluate on each element without knowing the underlying type.

```
class BasisFunction {
public:
    virtual double evaluate(double x) const = 0;
    ...
};
class LaguerrePolynomial : public BasisFunction {
public:
    double evaluate(double x) const override;
    ....
};
```

override keyword instructs the compiler to raise an error rather than introducing an unrelated new virtual method. This design makes the codebase extensible: adding a new basis function requires only writing a new subclass, without changes to the pricing logic.

[Polymorphism is realised through virtual dispatch of evaluate(). The regression code iterates over BasisFunction pointers and evaluates each term without knowledge of the underlying subclass, so the same routine works for all supported basis families.]

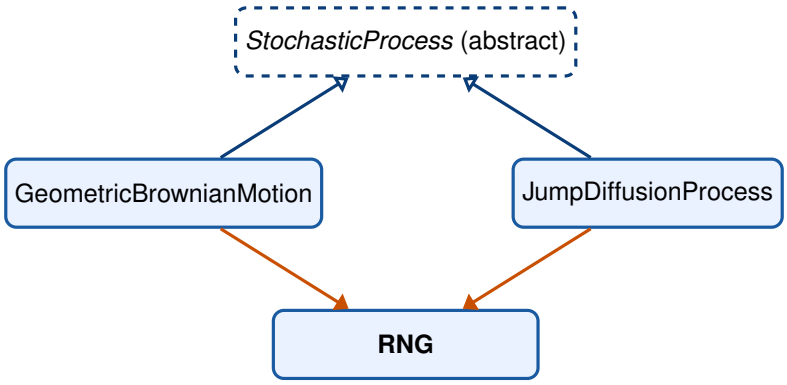
Encapsulation

Consistently applied across the module. MonomialBasis and LaguerrePolynomial hold private members (power_ and order_) validated at construction time. ConstantBasis has no state and BasisSet correctly encapsulates its internal storage as a **private** member:

```
class BasisSet {
private:
    std::vector<std::unique_ptr<BasisFunction>> basis_;
```

Encapsulation is provided through private storage of basis-specific parameters and through BasisSet, which owns a private std::vector<std::unique_ptr<BasisFunction». External code accesses the collection only through controlled methods, preserving internal ownership and construction logic.

C.3 Underlying SDE



The underlying SDE module provides the stochastic dynamics used to generate simulated asset paths for the LSM algorithm. A common abstract interface is defined through `StochasticProcess`, while concrete subclasses implement the specific one-step dynamics for GBM and jump-diffusion models. This allows the simulation engine to work with different models through the same interface.

Abstraction

`StochasticProcess` defines a common interface through two key methods: a pure virtual one-step update, and a concrete path simulator built on top.

```
virtual double stepWithNormal(double s, double dt, double z, RNG& rng) const = 0;
std::vector<double> simulatePath(double S0, double T, std::size_t n, RNG& rng) const;
```

Since `stepWithNormal` is pure virtual, every derived class must supply its own dynamics, while the general simulation logic in `simulatePath` is written once in the base class.

Inheritance

`GeometricBrownianMotion` and `JumpDiffusionProcess` both derive publicly from `StochasticProcess`, inheriting the shared simulation framework while specialising the one-step dynamics where necessary.

```
class GeometricBrownianMotion final : public StochasticProcess { ... };
class JumpDiffusionProcess final : public StochasticProcess { ... };
```

`JumpDiffusionProcess` additionally overrides `step`, since jump detection requires an extra uniform random draw beyond the diffusion shock.

```
double step(double s, double dt, RNG& rng) const override;
```

Polymorphism

Pricing and simulation code operates on the base type `StochasticProcess`, with the correct model-specific dynamics resolved at runtime. GBM update using the standard log-normal step:

```
// GeometricBrownianMotion
```

```
return s * std::exp((r_ - 0.5*sigma_*sigma_)*dt + sigma_*std::sqrt(dt)*z);
```

while the jump-diffusion update first checks for a jump event before applying diffusion,

```
// JumpDiffusionProcess
if (rng.uniform01() < 1.0 - std::exp(-lambda_*dt)) return 0.0;
return s * std::exp(((r_+lambda_) - 0.5*sigma_*sigma_)*dt + sigma_*std::sqrt(dt)*z);
```

A call through a `StochasticProcess` pointer dispatches to the correct implementation depending on the underlying subclass.

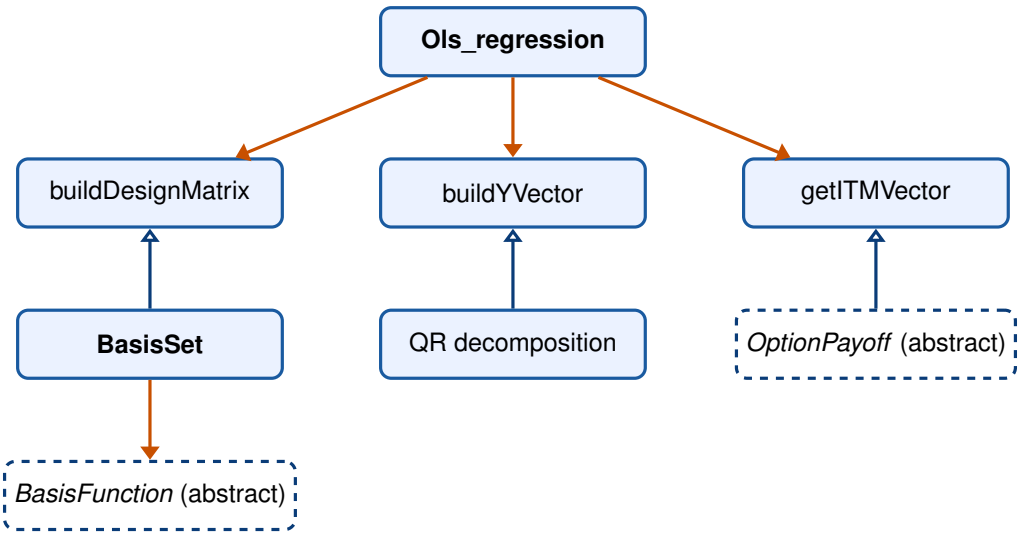
Encapsulation

Model parameters are stored as private members and exposed only through const getter methods.

```
double r() const { return r_; }
double sigma() const { return sigma_; }   // GeometricBrownianMotion
double lambda() const { return lambda_; } // JumpDiffusionProcess (additional getter)
```

Encapsulation is weaker in the `RNG` struct, however, where the engine and distributions are public members, meaning external code can directly modify the generator state.

C.4 OLS Regression



Abstraction

Composed of modular functions that each focus on specific task:

- `getITMVector` decides *which* paths are ITM, delegating the actual test to the polymorphic `OptionPayoff::InTheMoney`method. The caller never needs to know whether the option is a put or call.
- `buildDesignMatrix` translates a vector of spot prices into an Eigen matrix of basis-function evaluations, looping over the abstract `BasisFunction` pointers. The function is agnostic to which basis family (monomial, Laguerre, . . .) is in use.
- `buildYVector` filters and discounts future cashflows for ITM paths only.

- `Ols_regression` assembles the three helpers and delegates the linear solve to Eigen's column-pivoted Householder QR decomposition.

```
Eigen::VectorXd beta = X.colPivHouseholderQr().solve(Y);
// Solve OLS via QR: beta = (X'X)^{-1} X'Y
```

By deferring the numerical algebra to Eigen, the function cleanly separates the financial logic (which paths to include, how to normalize spot prices) from the numerical linear-algebra implementation. Additionally, it simplifies the code and ensures the accuracy of the OLS regression with QR decomposition (see GitHub pages on OLS regression for details).

Inheritance

This module contains no class definitions of its own and therefore introduces no inheritance hierarchy directly. It is composed of free functions grouped under `lsm::engine`. However, it uses the inheritance hierarchies defined in `option_payoff.hpp` and `basis_functions.hpp` since the functions accept references to the abstract base types `OptionPayoff` and `BasisFunction`, relying on derived classes (`Put_payoff`, `Call_payoff`, `MonomialBasis`, `LaguerrePolynomial`, `ConstantBasis`) being substitutable at the call site. The regression engine is, therefore, independent of any specific inheritance class: it was written against the base-class interfaces and works correctly with any future derived type without modification.

Polymorphism

Although this module does not define any virtual classes, it uses polymorphisms defined elsewhere.

BasisFunction polymorphism. `buildDesignMatrix` iterates over the `BasisSet::basis` vector of `std::unique_ptr<BasisFunction>`, calling `evaluate` through the base-class pointer, but C++ picks the right derived `evaluate()` at runtime (wether it is `ConstantBasis`, `MonomialBasis`, or `LaguerrePolynomial`):

```
for (int k = 0; k < K; ++k)
    X(row, k) = basis.basis[k]->evaluate(S_t[i] / strike);
```

The normalization `S_t[i] / strike` is applied before the polymorphic call, ensuring numerical stability regardless of the basis type.

OptionPayoff polymorphism. `getITMVector` calls `payoff_function.InTheMoney(S_t[i])`, which is a virtual method on `OptionPayoff`. Whether the concrete payoff is any type of options, this function requires no changes:

```
itm[i] = payoff_function.InTheMoney(S_t[i]);
```

Together, these two calls exhibit the modularity of the code design: adding a new basis family or a new payoff type requires zero modifications to the regression engine.

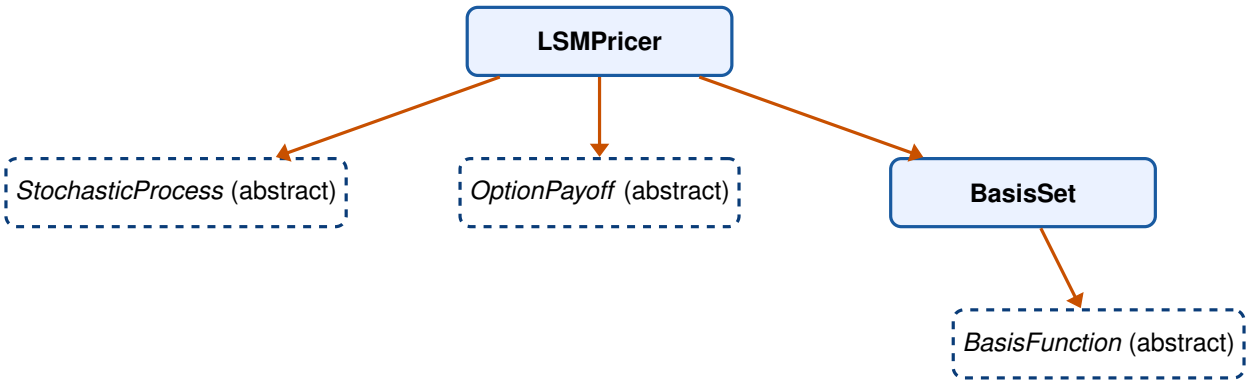
Encapsulation

Achieved through the decomposition of `Ols_regression` into three helper functions, each hiding the details of its step from the caller.

- `buildDesignMatrix` encapsulates the loop over ITM paths and the basis evaluation, exposing only an Eigen matrix to the caller. The row-counting and index-skipping logic for OTM paths is internal.
- `buildYVector` hides the discounting and filtering of cashflows. The caller does not require knowing that OTM entries are excluded from the output vector.
- `Ols_regression` encapsulates the regression coefficient vector `beta` entirely since it is a local variable that never escapes the function. The caller receives only the per-path continuation values `C_hat`, which is the quantity needed by the backward induction loop. This prevents any external code from depending on or misusing the intermediate coefficient vector.

The normalization `S_t[i] / strike` is encapsulated within the design-matrix and prediction steps, so the numerical scaling convention is enforced consistently and cannot be applied incorrectly by the caller.

C.5 LSM Pricer



C.5.1 Abstraction

Throughout `LSMPricer`, none of the member functions work directly with a concrete model. The constructor accepts references to `StochasticProcess`, `OptionPayoff`, and `BasisSet`, without storing their internal details:

```
LSMPricer::LSMPricer(
    const lsm::core::StochasticProcess& process,
    const lsm::core::OptionPayoff& payoff,
    const lsm::core::BasisSet& basis,
    const lsm::engine::LSMConfig& config)
: process(process), payoff(payoff), basis(basis), config(config)
{}
```

In `simulatePaths`, the path evolution calls `process.step` and `process.stepWithNormal` without knowing whether the underlying model is GBM or jump-diffusion.

```
path[t] = process.step(path[t - 1], dt, rng);
p1[t] = process.stepWithNormal(p1[t - 1], dt,  z, rng);
p2[t] = process.stepWithNormal(p2[t - 1], dt, -z, rng);
```

In `backwardInduction`, the regression step is delegated to `Ols_regression`. The function does not know how the design matrix is constructed or how the QR decomposition is performed internally. It passes in paths and cashflows, receiving continuation value estimates.

```
std::vector<double> coeffs = lsm::engine::Ols_regression(
    data.paths, static_cast<std::size_t>(t),
    cashflow, itm, discountFactor, basis, strike);
```

computeOptionValue receives a plain vector of present values and a European benchmark price, without knowledge of how those numbers were produced, and packages everything into a SimulationResult. Finally, price and priceWithData read as high-level descriptions of the algorithm, each sequencing the three steps of simulation, backward induction, and statistical computation.

```
PathData data = simulatePaths(S0);
std::vector<double> pv = backwardInduction(data);
return computeOptionValue(pv, europeanValue, N, T);
```

C.5.2 Inheritance

LSMPricer is written against base classes, and it is the inheritance hierarchy behind those base classes that allows the correct concrete behavior to be resolved at runtime.

StochasticProcess is the base class for all asset price models. GeometricBrownianMotion and JumpDiffusionProcess inherit from it and override step and stepWithNormal. Because simulatePaths only holds a reference to the base class, either derived model can be passed in without modifying simulation code.

OptionPayoff is the base class for option types, with Put_payoff and Call_payoff inheriting from it and implementing payoff() and InTheMoney(). Both backwardInduction and price call payoff.payoff() through this base class reference, so the exercise logic works identically for puts and calls without any branching on option type.

BasisFunction is the base class for individual regression terms. LaguerrePolynomial, MonomialBasis, and ConstantBasis all inherit from it and implement evaluate(). BasisSet holds a collection of these through base class pointers, meaning Ols_regression can evaluate whichever basis family was chosen without changing the regression code.

C.5.3 Polymorphism

In simulatePaths, every path evolution step goes through a virtual call on process.

```
path[t] = process.step(path[t - 1], dt, rng);
```

If the stored process is a GeometricBrownianMotion, this dispatches to the GBM log-normal update. If it is a JumpDiffusionProcess, it dispatches to the compensated drift version that also samples a jump event from the uniform distribution. The simulation loop itself is identical in both cases. For antithetic variates, the same polymorphism applies through stepWithNormal.

```
p1[t] = process.stepWithNormal(p1[t - 1], dt,  z, rng);
p2[t] = process.stepWithNormal(p2[t - 1], dt, -z, rng);
```

In backwardInduction and price, the payoff is evaluated through a virtual call on the OptionPayoff reference.

```
cashflow[i] = payoff.payoff(data.paths[i][numTimes]);
euSum += payoff.payoff(data.paths[i][T]);
```

At runtime, this dispatches to `Put_payoff::payoff`, returning $\max(K-S,0)$, or `Call_payoff::payoff`, returning $\max(S-K,0)$. The backward induction logic never checks which option type it is working with. The same applies to the ITM check used to filter paths before regression.

```
double exerciseValue = payoff.payoff(data.paths[i][t]);
if (exerciseValue > 0.0) { itm[i] = true; }
```

Inside `Ols_regression`, the basis functions are evaluated polymorphically through `BasisFunction` pointers held inside `BasisSet`. Whether the basis is Laguerre or monomial, the call `ptrs[k]->evaluate(...)` resolves to the correct implementation, so the regression is indifferent to the basis type chosen.

C.5.4 Encapsulation

The private members `process`, `payoff`, `basis`, and `config` are inaccessible to any caller of the class.

```
private:
    const lsm::core::StochasticProcess& process;
    const lsm::core::OptionPayoff& payoff;
    const lsm::core::BasisSet& basis;
    lsm::engine::LSMConfig config;
```

The three main steps of the algorithm simulation, backward induction, and statistics computation are each encapsulated as private member functions. The caller of `price()` or `priceWithData()` never directly invokes any of them.

```
private:
    lsm::engine::PathData simulatePaths(double S0) const;
    std::vector<double> backwardInduction(lsm::engine::PathData& data) const;
    lsm::engine::SimulationResult computeOptionValue(
        const std::vector<double>& pv,
        double europeanValue, int N, int T) const;
```

Within `backwardInduction`, the zeroing of future cashflows when early exercise is detected is an internal mutation of `PathData` that the caller does not need to manage:

```
for (int s = t + 1; s <= numTimes; ++s)
    data.cashFlows[i][s] = 0.0;
```

The numerical configuration is encapsulated inside `LSMConfig` rather than passed as individual parameters, meaning that adding or changing a simulation parameter does not require modifying any function signatures:

```
const int T = config.numExerciseDates;
const double dt = config.maturity / static_cast<double>(T);
double discountFactor = std::exp(-config.riskFreeRate * dt);
```

The public interface exposed to the caller is minimal: `price()` returns a `SimulationResult`, and `priceWithData()` additionally returns the `PathData`. Everything else like how paths are generated, how regression is performed, how early exercise decisions are made, is hidden inside the class.

C.6 Convergence Analyser

The `ConvergenceAnalyser` class is defined in `convergence_analyser.hpp` and `convergence_analyser.cpp`. It was made to take measurements of the LSM pricing engine across varying parameter settings, to assess both the accuracy and stability of the implemented LSM pricer.

Abstraction

The class has six public run functions: `runBenchmark()`, `runPathConvergence()`, `runDatesConvergence()`, `runOrderConvergence()`, `runFDConvergence()`, and `runSeedStability()`. These hide the complexity of constructing and running the full LSM pipeline. The helper function `getLSMPrice()` abstracts away the entire pricer setup, including process creation, basis population, and configuration, behind a single call that returns one value:

```
double ConvergenceAnalyser::getLSMPrice(unsigned seed, int numExerciseDates,
                                        int order, int numPaths) {
    lsm::engine::LSMConfig config;
    config.rngSeed          = seed;
    config.numExerciseDates = numExerciseDates;
    config.numPaths         = numPaths;
    config.riskFreeRate     = r;
    config.maturity         = T;

    lsm::core::BasisSet basis;
    basisFactory(basis, order);
    auto process = sdeFactory(r, sigma);
    lsm::engine::LSMPricer myPricer(*process, payoffType, basis, config);
    return myPricer.price(S0).optionValue;
}
```

Inheritance

`ConvergenceAnalyser` does not inherit from any base class, as there is no natural parent class for this standalone analysis tool. Instead, it acts as a client of the inheritance hierarchies defined elsewhere. The payoff object is passed in at construction as a `const OptionPayoff&`, so `getLSMPrice()` works correctly for any derived payoff type, put or call, without any branching logic inside the analyser itself.

Polymorphism

Inside `getLSMPrice()`, the stochastic process is constructed via `sdeFactory`, which returns a `std::unique_ptr<lsm` The pricer receives only a reference to the base class, so the correct one-step dynamics are resolved at runtime regardless of which model was supplied:

```
auto process = sdeFactory(r, sigma);
lsm::engine::LSMPricer myPricer(*process, payoffType, basis, config);
```

Similarly, `payoffType` is held as a `const OptionPayoff&`. Calls to `payoff()` and `InTheMoney()` dispatch virtually to the correct derived implementation — `Put_payoff` or `Call_payoff` — without the

analyser needing to know which type it is working with.

Encapsulation

All market parameters and factory functions are stored as private members making it inaccessible to the caller:

```
private:
    double S0, r, sigma, K, T;
    std::function<std::unique_ptr<lsm::core::StochasticProcess>(double, double)> sdeFactory;
    const lsm::core::OptionPayoff& payoffType;
    const lsm::core::BasisSet&     basisType;
    std::function<void(lsm::core::BasisSet&, int)> basisFactory;
    int order, fixedPathCount, fixedNumDates;
```

The helper `paramString()` is also private, used internally to generate consistent file name suffixes for `csv_output/`. Inside `getLSMPrice()`, the `LSMConfig`, `BasisSet`, and process objects are all local variables that never escape the function. The caller receives only the final option value, with no visibility into how the pricer was constructed or run.

D Implementation Issues

D.1 Eigen Package

An issue we encountered during development was with the **Eigen** linear algebra library. While the build works locally, the GitHub autograding environment does not contain Eigen packages, which caused compilation failure.

The fix we took was straightforward. We bundled the Eigen headers directly into the repository under the dependencies folder (`include/Eigen/`), and updated `CMakeLists.txt` to point to it:

```
include_directories(${CMAKE_SOURCE_DIR}/include)
```

Since Eigen is a header-only library, this required no compilation or package installation, and the rest of the build pipeline stayed the same.

We considered different approaches before settling on this idea. One option was to install Eigen through the autograding bash script using `sudo apt-get install -y libeigen3-dev`, but this would rely on the package being available on the autograding machine, which we could not guarantee. Another option was using CMake's `FetchContent` to pull Eigen from its GitLab repository at build time, but this introduces a network dependency and slows down the build, which was not ideal for an autograding setup.

Bundling Eigen directly in the repository turned out to be the simplest and most reliable solution, as it keeps the build fully self-contained and reproducible across any environment and operating system.

D.2 Compiling Times

During the project files building, while individual testing, there is an inefficiency in the build system caused by every source file in the library being recompiled once per executable target. Since the project defines multiple test binaries alongside two main executables, each `.cpp` file in `src/` was being compiled five to eight times per build, which significantly inflates CI pipeline times.

The root cause was listing the shared source files directly in each `add_executable` call in CMake as,

```
set(SRC_FILES src/basis_functions.cpp
    ...
)

add_executable(my_program ${SRC_FILES} src/main.cpp)
add_executable(test_lsm_pricer ${SRC_FILES} tests/test_lsm_pricer.cpp)
...
# each target above recompiles all of SRC_FILES independently
```

The CMakeList documentation was referred [6] and resolved this problem by extracting all shared sources into a single static library target defined at the root `CMakeLists.txt` level, with all executables and test targets linking against it.

```
add_library(lsm_core STATIC ${SRC_FILES})
target_include_directories(lsm_core PUBLIC
    ${CMAKE_SOURCE_DIR}/src
    ${CMAKE_SOURCE_DIR}/dependencies/eigen/5.0.1/include/eigen3
)
```

```cmake

add_executable(my_program src/main.cpp)
target_link_libraries(my_program PRIVATE lsm_core)

add_executable(convergence_tool src/generate_convergence_output.cpp)
target_link_libraries(convergence_tool PRIVATE lsm_core)
```

The same pattern was applied in `tests/CMakeLists.txt`, where a helper function was used to reduce redundancy as,

```cmake
function(add_lsm_test target source)
    add_executable(${target} ${source})
    target_include_directories(${target} PRIVATE
        ${CMAKE_SOURCE_DIR}/extern/catch2)
    target_link_libraries(${target} PRIVATE lsm_core)
    add_test(NAME ${target} COMMAND ${target})
endfunction()
```

Every single source file currently compiles precisely once, regardless of how many targets refer to it, and hence reduces the total build time upto 1 minute 40 seconds.

D.3 GitHub Upstream link

Throughout the project, our repository was configured with two remotes, `origin` that points to the team's individual repository, and `upstream` which points towards the original assignment template at `ib9jho-2025-2026-group-assignment-group-project-template.git`. We routinely ran `git fetch -all -tags -prune` to keep our local repository in sync, as this is, in generally a good practice within a collaborative setting to avoid working with stale branch references.

This routine sync was failing with a fatal error, `remote: Repository not found`. We found that the upstream remote was still pointing towards the original forked template repository, which was private and inaccessible. Rather than removing the upstream remote entirely, the correct fix was to update it to point to our own accessible repository, as it should be, to make the repository functional again.

E Variables Types, C++ Language Features and Structures

In this section, we discuss the usage of C++ features, structures, and variables along with the reasoning for their usage within our project. The usage can be seen in several parts of the projects, but for our discussion, we focus on the following,

`main.cpp` **and** `basis_functions.cpp`

E.1 constexpr

In `main.cpp`, `constexpr` is mainly used to define configuration parameters as,

```cpp
namespace cfg {
    // Market parameters
    constexpr double initial_spot = 1.0;
    ....
    ....
    // Output
```

```
    constexpr const char* output_dir = "../csv_output";
}
```

The idea is that these values never change, so it makes sense to fix them at compile time rather than setting them at runtime.

- The compiler evaluates them early, so there is no runtime overhead.
- They are guaranteed to stay constant, which avoids accidental changes.
- The compiler can optimise better since it knows the values upfront.
- They can also be used in places where true compile-time constants are required, like array sizes or template parameters.

E.2 auto

`auto` is used across the project files to let the compiler figure out the type:

```
auto ptrs = b.basisPtrs();
auto f = utils::open_file("paths.csv");
auto basis = std::make_unique<lsm::core::BasisSet>();
auto [result, path_data] = pricer.priceWithData(...);
```

In practice, this just makes the code easier to work with.

- It avoids writing long and messy types, especially with templates.
- If a function return type changes later, you do not need to update everything.
- It reduces the risk of type mismatches.
- It keeps the focus on what the variable represents, not its exact type.

E.3 static_cast

`static_cast` is used when we want an explicit and safe type conversion:

```
Eigen::VectorXd Y = Eigen::Map<Eigen::VectorXd>(
    y_vec.data(), static_cast<int>(y_vec.size()));
```

Here, the conversion is intentional. `y_vec.size()` returns a `size_t`, but Eigen expects an `int`.

- It makes the conversion explicit, so it is clear to the reader.
- The compiler checks that the conversion is valid.
- It is safer than old C-style casts.
- It makes it easier to track where conversions happen in the code.

E.3.1 Lambda Expressions

```
Eigen::MatrixXd X = lsm::engine::buildDesignMatrix([&]{
        std::vector<double> s(np);
        for(int i = 0; i < np; ++i)
            s[i] = d.paths[i][ns - 1];
        return s;
    }(),
    itm_mask, basis, strike);
```

Lambdas are useful for short, local logic. Instead of defining a separate function, everything stays inline. Capturing by reference (`[&]`) also avoids unnecessary copies.

E.4 Explicit Constructors

```
explicit MonomialBasis(int power);
explicit LaguerrePolynomial(int order);
```

These constructors take a single argument, so marking them `explicit` avoids unintended implicit conversions.

- This prevents accidental construction from basic types like `int`.
- Makes object creation intentional and clear.
- Reduces risk of subtle bugs from implicit conversions.

E.5 Override Specifier

```
double evaluate(double x) const override;
std::string name() const override;
```

The `override` keyword on each subclass method provides a compile-time guarantee that the signature exactly matches the base class virtual.

E.6 Deleted Copy Operations

```
BasisSet(const BasisSet&) = delete;
BasisSet& operator=(const BasisSet&) = delete;
```

To avoids issues with copying `unique_ptr` while preventing incorrect shallow copies. So that, this makes it clear that the class owns resources exclusively.

E.7 Reserve for Performance

```
ptrs.reserve(basis_.size());
```

To allocates memory upfront and avoids repeated reallocation during `push_back` which improves performance in loops and numerical code.